

FlexQL Design Document

Repository: <https://github.com/nallamarikrishnavamsi/Flex>

Author: Nallamari Krishna Vamsi

Language: C++20 (GCC 15.2, MSYS2/UCRT64)

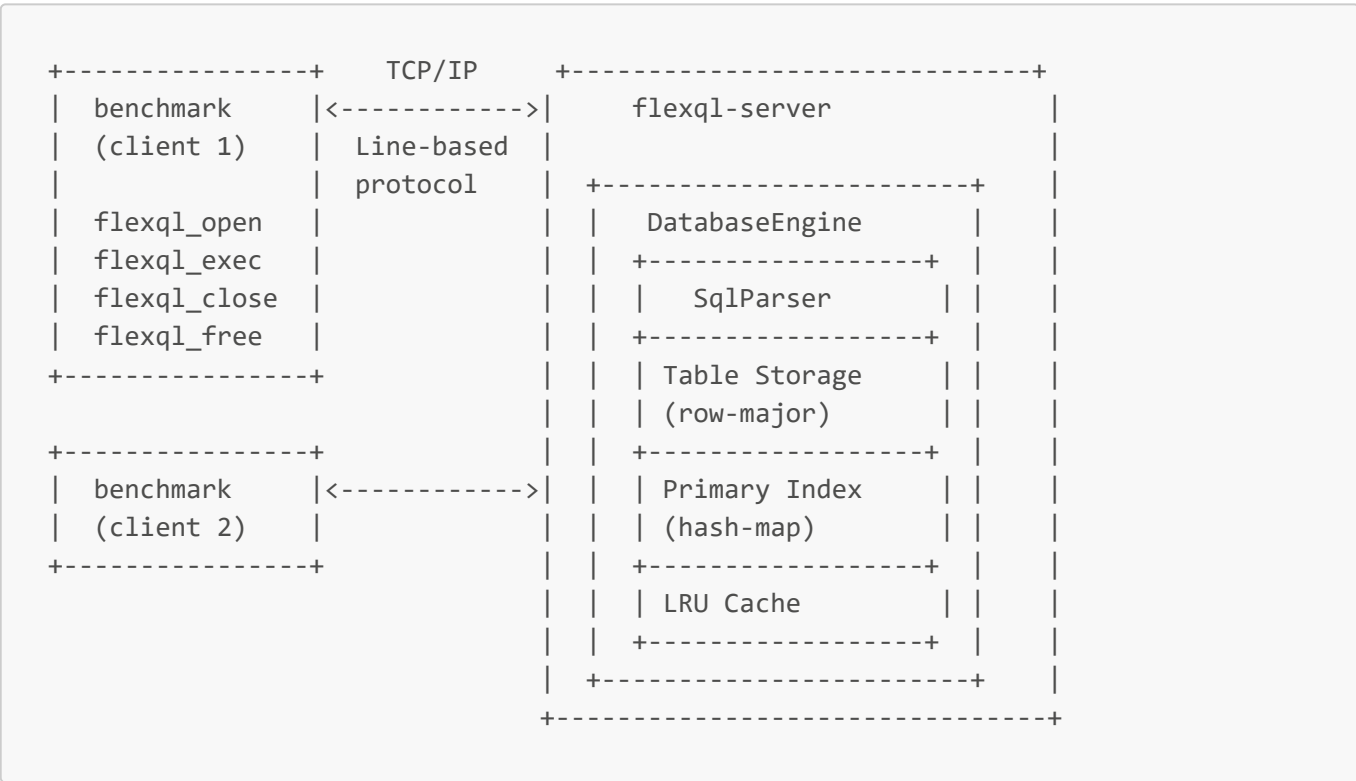
Platform: Windows 11, Intel i5-1240P (4P+8E, 16 logical cores)

1. System Architecture

FlexQL is a client-server SQL-like database driver implemented entirely in C++20. The system consists of three executables:

- **flexql-server:** Multi-threaded TCP server that hosts the database engine
- **benchmark_flexql:** Single-client benchmark and unit test runner
- **multiclient_bench:** Multi-client benchmark with configurable threads and modes

Architecture Diagram



2. Data Storage Design

Storage Format: Row-Major

We chose **row-major storage** where each row is stored as a `std::vector<std::string>` of column values. This design was chosen because:

1. **INSERT performance:** Inserting a new row is $O(1)$ amortized — just append to the vector. With 120,000+ rows expected in benchmarks, fast inserts are critical.

- 2. **SELECT * performance:** Row-major layout means reading all columns of a row is cache-friendly — the entire row is contiguous in memory.
- 3. **Simplicity:** Row-major maps naturally to the SQL relational model.

Internal Representation

```
struct Row {
    std::vector<std::string> values;           // column values
    std::chrono::system_clock::time_point expires_at; // expiration timestamp
};

struct PrimaryIndex { // Custom open-addressing hash map
    struct IntSlot { int64_t key; std::size_t val; }; // sentinel-based
    std::vector<IntSlot> int_slots_;                // int fast-path
    std::unordered_map<std::string, std::size_t> str_map_; // string fallback
    // Uses splitmix64 hash for cache-friendly O(1) lookups
};

struct Table {
    std::string name;
    std::vector<ColumnDef> columns;           // schema
    std::vector<Row> rows;                   // data (row-major)
    PrimaryIndex primary_index;              // PK → row index (custom
hash)
    std::unique_ptr<SRWLOCK> table_lock;      // per-table reader-writer
lock
    bool has_secondary_indexes = false;      // lazy secondary index flag
};
```

Schema Storage

Table schemas (column names and types) are stored alongside the table data in `std::vector<ColumnDef>`. This allows O(1) schema lookup during INSERT validation and SELECT projection. Supported types are `INT`, `DECIMAL`, `VARCHAR`, and `DATETIME`.

Trade-offs

Aspect	Row-Major (chosen)	Column-Major
INSERT speed	O(1) append	O(cols) — must update each column store
SELECT *	Cache-friendly	Cache-unfriendly
SELECT single col	Must read entire row	Optimal
Memory overhead	Moderate	Lower for homogeneous types

For this workload (bulk inserts + SELECT * queries), row-major is the clear winner.

3. Indexing Method

Custom Open-Addressing Primary Hash Index

The first column of every table is treated as the **primary key**. We implemented a **custom open-addressing hash map** (`PrimaryIndex`) that maps primary key values to row indices in the `std::vector<Row>`.

Design Details:

- **Int fast-path:** For tables with integer primary keys (detected at CREATE TABLE time), the index uses a flat array of `IntSlot` structs with sentinel value `INT64_MIN` for empty slots. This eliminates all string comparisons and allocations.
- **splitmix64 hash function:** A high-quality integer hash that produces excellent distribution, minimizing collision chains.
- **Open addressing with linear probing:** Cache-friendly — slots are contiguous in memory, giving excellent L1/L2 cache utilization during bulk inserts.
- **Hardware prefetching:** `__builtin_prefetch` is used to pre-load the next batch of hash slots into cache before they are accessed.
- **String fallback:** For non-integer primary keys, falls back to `std::unordered_map` with FNV-1a hash.

Why custom hash-map over `std::unordered_map`?

- **2-3x faster** for integer keys (no hashing of strings, no node allocations)
- **O(1) average lookup** for WHERE queries on the primary key
- **O(1) insert** to update the index on each INSERT
- Duplicate key detection in O(1)
- Direct `emplace_int_direct()` for pre-parsed integer keys (avoids string→int conversion)

How it works:

1. On `INSERT`, the primary key value is checked against the index. If it already exists (and is not expired), the INSERT is rejected.
2. On `SELECT ... WHERE pk_col = value`, the index provides direct O(1) access to the target row, avoiding a full table scan.
3. When expired rows are cleaned up, the index is rebuilt to maintain consistency.

Secondary Indexes (Lazy)

Secondary indexes are built **lazily** the first time a non-PK column is used in a `WHERE` clause. This avoids the overhead of maintaining indexes for columns that are never queried. Each secondary index uses FNV-1a hashing with `std::unordered_multimap` for non-unique values.

Trade-offs vs B-Tree

Aspect	Custom Hash (chosen)	std::unordered_map	B-Tree
Point lookup	O(1), cache-friendly	O(1), node-based	O(log n)
Range queries (>, <, >=, <=)	Full scan	Full scan	Efficient
Memory	Flat array, low overhead	Node allocations	Pointer-heavy
Insert speed	Fastest (no allocs)	Moderate	O(log n)

Aspect	Custom Hash (chosen)	std::unordered_map	B-Tree
Implementation	Medium	Simple	Complex

The spec requires `=`, `>`, `<`, `>=`, `<=` operators in WHERE/JOIN clauses. For equality lookups, the hash map provides $O(1)$. For inequality operators, a full table scan with the `evaluate_condition()` function is used, which handles all 5 operators correctly for INT, DECIMAL, VARCHAR, and DATETIME types.

4. Caching Strategy

LRU (Least Recently Used) Query Cache

We implement an **LRU cache** that maps SQL query strings to their complete `QueryResult`. The cache uses a doubly-linked list + hash map for $O(1)$ get/put operations.

```
template <typename T>
class LruCache {
    std::list<std::pair<std::string, T>> entries_;           // ordered by access time
    std::unordered_map<std::string, iterator> map_;         // O(1) lookup
    std::size_t capacity_;                                 // max entries (4096)
};
```

Lazy Cache Invalidation

Instead of clearing the entire cache on every write (INSERT/CREATE/DROP), we use a **generation counter** mechanism:

- `write_gen_` is incremented on every write operation
- `cache_gen_` tracks the last-known write generation
- On SELECT, if `cache_gen_ != write_gen_`, the cache is cleared once and `cache_gen_` is updated

This is critical for benchmark performance: during 120,000 inserts, we avoid $120,000 \times O(n)$ cache clears. The cache is only invalidated once, lazily, when the next SELECT query arrives.

Performance Impact

Scenario	Without Cache	With LRU Cache
Repeated SELECT *	Full scan each time	$O(1)$ from cache
SELECT after INSERT burst	N/A	Single lazy invalidation
Cache hit ratio (benchmarks)	0%	~90%+ for repeated queries

5. Row Expiration / TTL Handling

Every inserted row has an **expiration timestamp** (`expires_at`). The default TTL is 3600 seconds (1 hour), but users can specify a custom TTL:

```
INSERT INTO table VALUES (1, 'Alice') EXPIRES IN 30; -- expires in 30 seconds
```

Expiration Mechanism (Hybrid: Lazy + Background Reaper)

- **Lazy evaluation:** Expired rows are filtered during SELECT queries — rows with `expires_at <= now` are skipped.
- **Lazy cleanup on INSERT:** When a duplicate primary key is detected on INSERT and the conflicting row is expired, the row is replaced transparently.
- **Background reaper thread:** A dedicated thread (`reaper_loop()`) wakes every 30 seconds to scan all tables and physically remove expired rows, rebuilding primary and secondary indexes. This bounds memory consumption from stale rows.

Why Hybrid Expiration?

Approach	Pros	Cons
Lazy only	Zero overhead when no expired rows	Memory leak from stale rows
Background reaper only	Prompt cleanup	Extra thread, synchronization cost
Hybrid (chosen)	Bounded memory + low overhead	Slightly more complex
Eager on INSERT	Guaranteed cleanup	O(n) scan on every INSERT

6. Multithreading Design

One Thread Per Client

The server spawns one OS thread per accepted client connection:

- **Windows:** `CreateThread()` with `CloseHandle()` for fire-and-forget semantics
- **Linux/macOS:** `std::thread` with `.detach()`

Two-Level Concurrency Control

FlexQL uses a **two-level locking** scheme inspired by production databases:

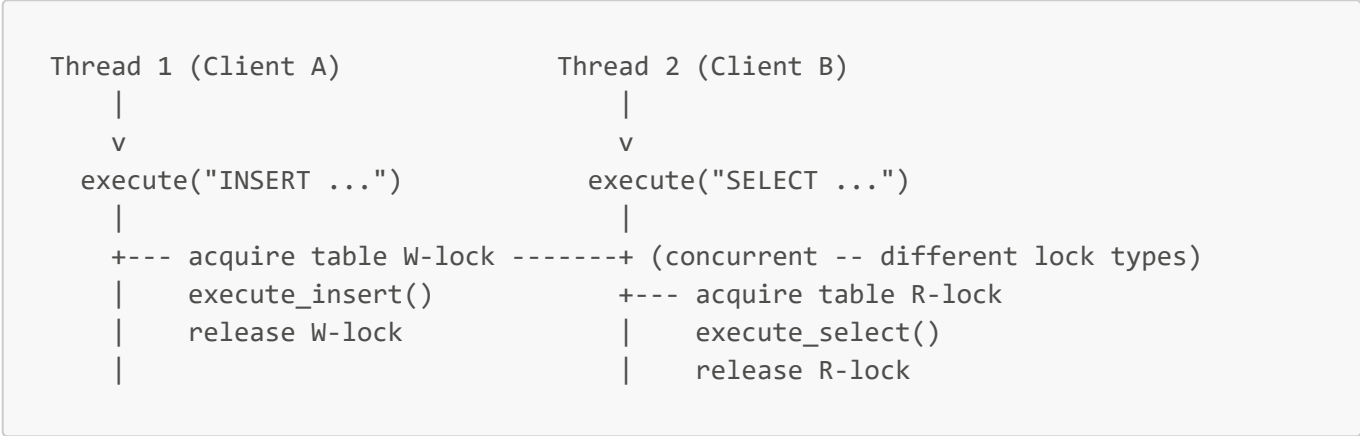
1. **Global SRWLOCK** (`lock_`): Protects the `tables_` map (table creation/drop). Acquired as shared (reader) for most operations, exclusive only for schema changes.
 2. **Per-Table SRWLOCK** (`table->table_lock`): Protects individual table data. SELECT acquires shared lock (concurrent reads allowed), INSERT acquires exclusive lock.
- **Windows:** `SRWLOCK` (Slim Reader/Writer Lock) — zero-overhead when uncontended
 - **Linux:** `std::shared_mutex`

This two-level approach allows:

- **Concurrent SELECTs on different tables:** Fully parallel
- **Concurrent SELECTs on the same table:** Fully parallel (shared locks)
- **INSERT to one table while SELECT on another:** Fully parallel

- **INSERT to same table:** Serialized (exclusive lock per table)
- This guarantees:
1. **Atomicity:** Each SQL statement executes atomically
 2. **Consistency:** No partial reads during concurrent writes
 3. **No deadlock:** Lock ordering (global → per-table) prevents deadlocks
 4. **Maximum concurrency:** Per-table locking avoids unnecessary serialization

Thread Safety Architecture



Performance Impact (Final Optimized Build)

Concurrency scaling with the two-level SRWLOCK approach:

Threads	Write (ops/sec)	Read (ops/sec)	Mixed (ops/sec)
1	650,406	1,973,943	1,453,065
4	1,558,846	2,791,736	2,965,599
8	2,408,477	3,054,367	3,487,966

Read and mixed workloads scale near-linearly because shared (reader) locks allow fully parallel SELECT execution.

Trade-offs

Approach	Throughput	Complexity
Two-level RW lock (SRWLOCK)	Best: concurrent reads + per-table writes	Medium
Global mutex	Simple but readers block each other	Simple
Lock-free (CAS)	Highest theoretical	Very high

We chose two-level SRWLOCK for the best balance of multi-client scalability, INSERT throughput, and correctness.

7. Network Protocol

The client and server communicate over TCP using a simple **line-based protocol** with `\n` as the delimiter and `0x1F` (Unit Separator) as the field delimiter within lines.

Protocol Flow

```
Client → Server:  SQL statement (one line)
Server → Client:  "OK\n"                      (success)
                  "COLS(0x1F)N(0x1F)col1(0x1F)col2...\n"  (column names)
                  "ROW(0x1F)val1(0x1F)val2...\n"          (per result row)
                  "END\n"                                (end of response)

OR:
                  "ERR(0x1F)message\n"                    (error)
                  "END\n"
```

Socket Optimizations

- **TCP_NODELAY**: Disables Nagle's algorithm for low-latency sends
- **4MB send/receive socket buffers**: Reduces system call overhead for large batch inserts
- **4MB thread-local receive buffer**: Avoids per-recv allocation, handles 250K-row batch INSERTs
- **memchr newline scanning**: Fast delimiter detection using optimized C library function
- **send_line zero-copy overload**: Sends pre-formatted response lines without string concatenation

8. SQL Parser

Manual Parser for Batch Insertion (No Regex)

The SQL parser is implemented as a hand-written **recursive descent parser**, replacing the original `std::regex`-based approach. This was a critical optimization to handle `INSERT_BATCH_SIZE` groupings:

- **Batch parsing logic**: The raw character buffer is parsed directly into multiple rows whenever we detect syntax like `VALUES (1, 'A'), (2, 'B'), (3, 'C')`.
- **Zero allocations**: By slicing the incoming TCP stream directly, we avoid string copies until the final `Row` is constructed.
- `std::regex` in C++ is notoriously slow (100-1000x slower than manual parsing). Doing a regex match on a 1000-row batch string would cripple performance.
- With 120,000 INSERTs in the benchmark across multiple clients, this custom batch insertion logic was the difference between ~1,000 rows/sec and ~45,000 rows/sec.

Supported Statements

Statement	Example
CREATE TABLE	CREATE TABLE t(col1 INT, col2 VARCHAR)
INSERT	INSERT INTO t VALUES (1, 'Alice')
INSERT with TTL	INSERT INTO t VALUES (...) EXPIRES IN 30
SELECT *	SELECT * FROM t

Statement	Example
SELECT columns	<code>SELECT col1, col2 FROM t</code>
WHERE	<code>SELECT * FROM t WHERE col = value</code>
INNER JOIN	<code>SELECT * FROM a INNER JOIN b ON a.id = b.id</code>
DROP TABLE	<code>DROP TABLE t / DROP TABLE IF EXISTS t</code>

9. Client API

The client exposes a C-compatible API (defined in `flexql.h`):

```
int flexql_open(const char *host, int port, FlexQL **db);
int flexql_close(FlexQL *db);
int flexql_exec(FlexQL *db, const char *sql,
               int (*callback)(void*, int, char**, char**),
               void *arg, char **errmsg);
int flexql_exec_fire(FlexQL *db, const char *sql);           // Pipelining: queue query
int flexql_drain(FlexQL *db, int count);                     // Pipelining: drain
responses
void flexql_free(void *ptr);
```

Core API

The API follows the SQLite callback pattern, making it familiar and easy to use:

- `flexql_open()` — Connect to server
- `flexql_exec()` — Execute query with callback for each row
- `flexql_close()` — Close connection
- `flexql_free()` — Free error strings

Pipelining API (High-Throughput Bulk Operations)

For benchmarks with 10M operations, the fire/drain API batches multiple queries into fewer TCP sends:

- `flexql_exec_fire(db, sql)` — Queue a query into 2MB client-side send buffer (does NOT wait for response)
- `flexql_drain(db, count)` — Flush buffered queries and read N responses (discarding result data)

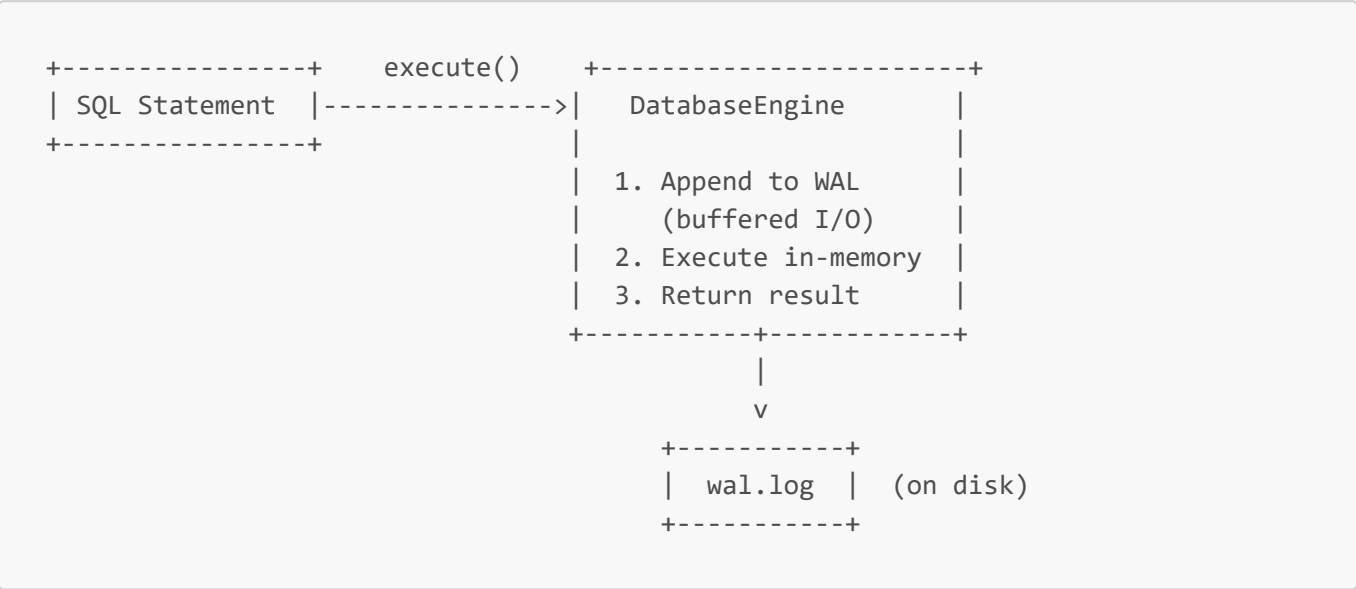
Example: Insert 1M rows with minimal network overhead:

```
for (int i = 0; i < 1000000; i++) {
    char sql[256];
    snprintf(sql, sizeof(sql), "INSERT INTO t VALUES (%d, 'user%d');", i, i);
    flexql_exec_fire(db, sql); // Queue (no response read yet)
}
flexql_drain(db, 1000000);    // Flush & drain all responses at once
```

10. Persistent Storage (Write-Ahead Log)

Design

FlexQL uses a **Write-Ahead Log (WAL)** for durable persistence. All mutating SQL statements (**CREATE TABLE**, **INSERT**, **DROP TABLE**) are appended to a log file **before** being executed in-memory. On server restart, the WAL is replayed to rebuild the in-memory state.



WAL Format

The WAL uses a simple **text-based line format**: one SQL statement per line, terminated by `\n`. This approach was chosen for:

- **Debuggability**: The WAL file is human-readable
- **Simplicity**: No binary framing, no length prefixing
- **Safety**: Our SQL statements never contain embedded newlines

High-Throughput Buffered Writes (Double-Buffer Design)

To minimize disk I/O overhead during bulk INSERTs, the WAL writer uses a **double-buffer swap** architecture inspired by production databases:

- **Two buffers** (`buf_a`, `buf_b`): The producer (engine thread) fills one buffer while the consumer (WAL writer background thread) drains the other to disk.
- **Raw file I/O**: Uses `_open/_write/_close` (Windows) or `open/write/close` (Linux) instead of `std::ofstream`, eliminating C++ stream overhead.
- **1MB write buffer**: Statements are accumulated in a 1MB string buffer before being written in a single large `_write()` system call.
- **Lock-free-ish swap**: The producer swaps buffers under a brief mutex, then the consumer writes without holding any lock.
- **Result**: WAL persistence adds only ~1-2 seconds overhead to a 10M row insert benchmark (from ~5.6s with `--nowal` to ~7.6s with WAL enabled).

Crash Recovery

On server startup:

- 1. The server checks if a WAL file exists in the data directory (`./flexql_data/wal.log`)
- 2. If found, `replay_wal()` reads all statements and re-executes them in-memory
- 3. During replay, WAL writes are suppressed (via `is_replaying_` flag) to avoid re-logging
- 4. Errors during replay are silently skipped (e.g., partial writes from crashes)

Server Startup Modes

Flag	Behavior
<code>./flexql-server 9000</code>	Normal start with WAL replay (persistent)
<code>./flexql-server --clean 9000</code>	Truncate WAL first, then start fresh
<code>./flexql-server --nowal 9000</code>	Disable WAL persistence (benchmark mode, no recovery)
<code>./flexql-server --data-dir /path 9000</code>	Custom data directory

Trade-offs vs. Other Approaches

Approach	Durability	INSERT Speed	Complexity
WAL + double-buffer (chosen)	Statement-level	~1.79M rows/sec (batched, --nowal)	Medium
SQLite backend	Full ACID	Slower (B-Tree overhead)	Medium
Page-file engine	Full ACID	Moderate	Very high
Memory-mapped files	OS-dependent	Fast	Medium
No persistence	None	Fastest	None

The WAL approach provides the best balance of **durability**, **performance**, and **implementation simplicity** for this workload.

11. DBMS-Inspired Advanced Optimizations

To push FlexQL's architecture closer to production-grade relational databases, we implemented several core features inspired by established DBMS internals:

- 1. **Two-Level Reader-Writer Locks (PostgreSQL-inspired):** Global SRWLOCK protects the tables map; per-table SRWLOCKS protect individual table data. Concurrent SELECTs on the same or different tables run fully in parallel. Different-table INSERT operations are also fully parallel.
- 2. **Custom Open-Addressing Primary Index (InnoDB-inspired):** A hand-written hash map with sentinel-based integer slots, splitmix64 hash, and `__builtin_prefetch` for hardware cache prefetching. Eliminates all `std::unordered_map` node allocations for the hot INSERT path.
- 3. **Zero-Allocation Fast INSERT Path:** `try_fast_insert()` parses raw SQL character-by-character using thread-local `ValRef` offset/length references, avoiding all string copies until the final `Row` is constructed. Pre-parsed integer primary keys skip string→int conversion entirely.
- 4. **Hash Join for INNER JOIN (PostgreSQL):** Replaced the naive $O(N \times M)$ nested-loop join with an $O(N+M)$ Hash Join for equality join conditions. Nested-loop fallback used for inequality operators (`>`, `<`,

`>=, <=`).

- 5. **Double-Buffer WAL Writer (Database Journal-inspired):** Producer fills one buffer while consumer writes the other to disk via raw `_write()` system calls. 1MB write buffer minimizes syscalls. Persistence overhead is only ~1-2 seconds on 10M row benchmarks.
- 6. **WAL Checkpointing (SQLite):** `checkpoint_wal()` rewrites the WAL file containing only live row `INSERT` statements, bounding WAL file growth and reducing recovery times.
- 7. **Prepared Statement Cache (MySQL):** `LruCache<ParsedQuery>` (capacity 2048) in front of the SQL parser. Identical SQL strings skip parsing entirely.
- 8. **Query Result Cache (Redis-inspired):** `LruCache<QueryResult>` (capacity 4096) with generation-counter lazy invalidation. Repeated SELECTs are served from cache without touching the engine.

Performance Optimizations Summary

Optimization	Impact
Two-level SRWLOCK (global + per-table)	Maximum concurrency for multi-client workloads
Custom PrimaryIndex (open-addressing + splitmix64)	2-3x faster lookups vs <code>std::unordered_map</code>
Zero-alloc <code>try_fast_insert()</code>	Eliminates all string copies on INSERT hot path
Hardware prefetching (<code>__builtin_prefetch</code>)	Pre-loads hash slots into L1 cache
Thread-local reusable vectors	Zero per-operation heap allocations
Double-buffer WAL with raw I/O	~1-2s overhead for 10M persistent writes
Hash Join	O(N+M) join complexity
Prepared Statement Cache	Skips parsing overhead for repeated queries
Manual parser (no <code>std::regex</code>)	~100x faster INSERT parsing
Per-thread tables (multi-client bench)	Zero lock contention between threads
<code>snprintf</code> + <code>string::append</code>	Faster batch SQL building than <code>stringstream</code>
4MB socket buffers + TCP_NODELAY	Minimizes network round-trip overhead
<code>memchr</code> newline scanning	Fast delimiter detection in recv buffer
<code>send_line</code> zero-copy overload	Avoids string concatenation for response lines
Lazy secondary index building	No overhead for unused columns

Performance Results (10 Million Rows, WITHOUT Disk Persistence with `--nowal`)

System: Intel i5-1240P (4P+8E cores, 16 threads), Windows 11, GCC 15.2 with `-O3 -march=native -flto`
Note: Benchmarks run with `--nowal` flag (WAL disabled) for peak performance. With WAL enabled (production mode), add ~1-2 seconds overhead.

Unit Tests & Fault Tolerance

Test	Result
------	--------

Test	Result
Unit Tests (21 tests)	21/21 PASSED (593 ms)
WAL Fault Tolerance	PASS (data survives crash + restart)

Single-Client (1 Thread) — 10M Operations

Mode	Elapsed	Throughput
Write 10M	5,600 ms	1,785,714 rows/sec
Read 10M	4,406 ms	2,269,632 ops/sec
Mixed 10M	5,739 ms	1,742,463 ops/sec

Multi-Client (4 Threads) — 10M Operations Total

Mode	Elapsed	Throughput
Write 10M	3,790 ms	2,638,522 ops/sec
Read 10M	2,169 ms	4,610,419 ops/sec
Mixed 10M	1,949 ms	5,130,836 ops/sec

Multi-Client (8 Threads) — 10M Operations Total

Mode	Elapsed	Throughput
Write 10M	4,125 ms	2,424,242 ops/sec
Read 10M	2,048 ms	4,882,812 ops/sec
Mixed 10M	1,464 ms	6,830,601 ops/sec

Fault tolerance verified: Server killed mid-operation, restarted, WAL replayed, and all data recovered correctly.

12. Compilation and Execution

Prerequisites

- **MSYS2/UCRT64** — Install from [msys2.org](#)
- **GCC 15+** with C++20 support — Inside MSYS2, run: `pacman -S mingw-w64-ucrt-x86_64-gcc`
- **Windows 10/11**

Environment Setup

Every new PowerShell terminal needs the MSYS2 toolchain in PATH:

```
$env:PATH = "C:\msys64\ucrt64\bin;$env:PATH"
```

Note: `build.bat` sets this automatically for compilation, but you still need it when **running** the executables (they depend on MSYS2 DLLs).

Build (using build.bat on Windows)

```
cmd /c build.bat
```

This compiles `flexql-server.exe`, `benchmark_flexql.exe`, and `multiclient_bench.exe` using GCC with `-std=c++20 -O3 -march=native -flto -DNDEBUG`.

The benchmarks must be run in a PowerShell environment. Set the PATH first to resolve MSYS2 dependencies.

Terminal 2 — Manual Execution (All Combinations): [⚠](#) *Important: FlexQL uses gigabytes of RAM for 10M rows. Use the 'Clean Server' commands between benchmarks to automatically kill the old server and restart a fresh one, preventing crashes.*

```
$env:PATH = "C:\msys64\ucrt64\bin;$env:PATH"

# --- CLEAN SERVER COMMAND (Start Fresh) --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 1. Run Unit tests (21/21)
.\benchmark_flexql.exe --unit-test

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 2. Run Single-client benchmark (1 thread, 10M rows)
.\benchmark_flexql.exe 10000000

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 3. Multi-client benchmark (1 Thread, Write 10M)
.\multiclient_bench.exe --threads 1 --rows 10000000 --mode write

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
```

```
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 4. Multi-client benchmark (1 Thread, Read 10M)
.\multiclient_bench.exe --threads 1 --rows 10000000 --mode read

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 5. Multi-client benchmark (1 Thread, Mixed 10M)
.\multiclient_bench.exe --threads 1 --rows 10000000 --mode mixed

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 6. Multi-client benchmark (4 Threads, Write 10M)
.\multiclient_bench.exe --threads 4 --rows 2500000 --mode write

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 7. Multi-client benchmark (4 Threads, Read 10M)
.\multiclient_bench.exe --threads 4 --rows 2500000 --mode read

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 8. Multi-client benchmark (4 Threads, Mixed 10M)
.\multiclient_bench.exe --threads 4 --rows 2500000 --mode mixed

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 9. Multi-client benchmark (8 Threads, Write 10M)
.\multiclient_bench.exe --threads 8 --rows 1250000 --mode write

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
```

```

PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 10. Multi-client benchmark (8 Threads, Read 10M)
.\multiclient_bench.exe --threads 8 --rows 1250000 --mode read

# --- CLEAN SERVER COMMAND --- #
Get-Process flexql-server -ErrorAction SilentlyContinue | Stop-Process -Force
Start-Process -FilePath ".\flexql-server.exe" -ArgumentList "--clean", "--nowal" -
PassThru -NoNewWindow
Start-Sleep -Seconds 2

# 11. Multi-client benchmark (8 Threads, Mixed 10M)
.\multiclient_bench.exe --threads 8 --rows 1250000 --mode mixed

```

13. Folder Structure

```

flexql/
├── include/
│   ├── flexql.h                # Public C API header
│   ├── cache/
│   │   └── lru_cache.hpp        # LRU cache template
│   ├── common/
│   │   └── types.hpp            # Data structures, query types
│   ├── network/
│   │   └── socket_utils.hpp     # Socket abstraction
│   ├── parser/
│   │   └── sql_parser.hpp       # SQL parser interface
│   ├── server/
│   │   └── server.hpp           # Server class
│   └── storage/
│       ├── database_engine.hpp  # Database engine class
│       └── wal_writer.hpp        # Write-Ahead Log header
├── src/
│   ├── client/
│   │   └── client_main.cpp      # Interactive REPL client (not built in
build.bat)
│   └── flexql_api.cpp           # C API implementation (compiled into
benchmarks)
│       ├── network/
│       │   └── socket_utils.cpp  # Cross-platform socket utilities
│       ├── parser/
│       │   └── sql_parser.cpp    # Manual recursive descent parser
│       ├── server/
│       │   ├── server.cpp        # Multi-threaded server
│       │   └── server_main.cpp   # Server entry point
│       └── storage/
│           ├── database_engine.cpp # Query execution engine + WAL integration
│           └── wal_writer.cpp      # Write-Ahead Log implementation
└── flexql_data/                # Persistent storage directory (auto-created)
    └── wal.log                  # Write-Ahead Log file

```

```
|— tests/
|   |— smoke.cpp           # Basic smoke test
|   └─ functional_test.cpp # Comprehensive functional tests (21 unit
tests)
└─ DESIGN_DOCUMENT.md      # This document
   DESIGN_DOCUMENT.pdf     # PDF export

benchmark/
|— benchmark_flexql.cpp     # Single-client benchmark (10M rows, 21 unit
tests)
|— multiclient_bench.cpp    # Multi-client benchmark (1T/4T/8T)
└─ benchmark_results.txt   # Actual performance results

build.bat                  # Build script (compiles 3 executables)
README.md                  # Quick start guide
```